

Projet Pratique 2 : Gestion des Inscriptions en Ligne

Table des Matières

1.Documentation

Technique

- 1.1.Modèle Conceptuel de Données (MCD)
- 1.2. Modèle Logique de Données (MLD)
- 1.3. Schéma de la Base de Données
- 1.4. Documentation des Tables

2.Scripts

SQL

- 2.1. Scripts de Création et d'Insertion
- 2.2. Scripts de Requêtes
- 2.3. Scripts de Procédures et Fonctions Stockées
- 2.4. Scripts de Vues et Triggers

3. Rapport de Projet : Gestion des Inscriptions en Ligne pour la Formation KFOKAM48

- 3.1.Introduction
- 3.2. Méthodologie

- Analyse des Entités
- Définition des Tables et Attributs
- Gestion des Contraintes et Clés
- Optimisation des Performances
- Gestion de l'Histoire

4.Résultats

- a. Création de la Base de Données et des Tables
- b. Insertion des Données Fictives dans la Table Formations
- c. Requête : Lister les Étudiants Inscrits à une Formation
- d. Création d'une Procédure Stockée d'Inscription
- e. Création d'un Trigger pour la Mise à Jour du Statut de Paiement

5. Conclusion

1. Documentation Technique

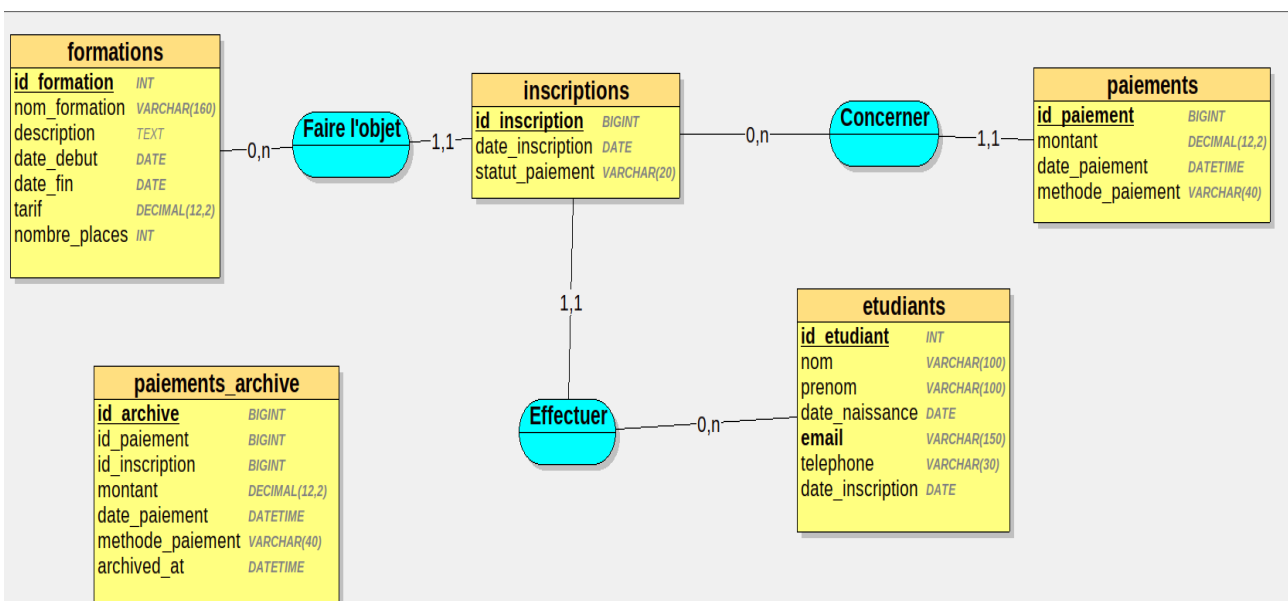
1-1-Modèle conceptuel (MCD)

Le MCD représente les entités et leurs relations. Il sert de base pour la conception de la base de données relationnelle.

- Étudiant : Un étudiant est une personne inscrite.
- Formation : Une formation est un programme d'étude.
- Inscription : L'inscription est une association entre un étudiant et une formation.
- Paiement : Un paiement est associé à une inscription.

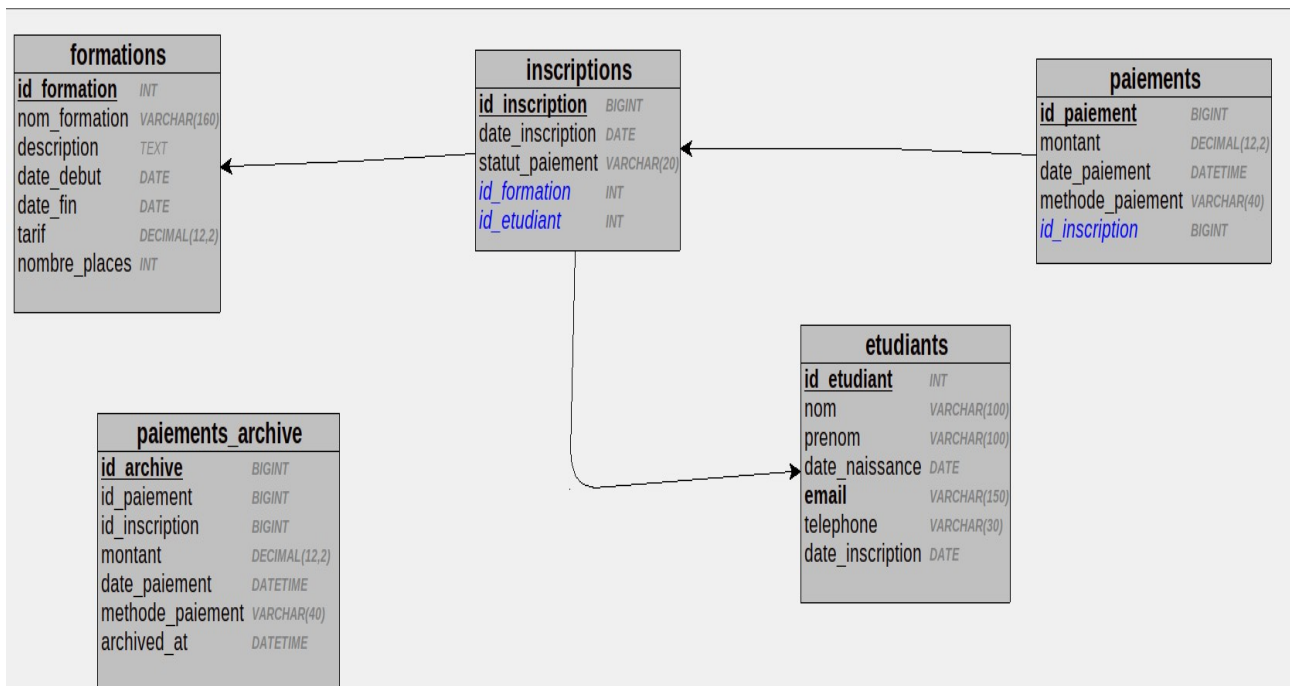
Relations :

- Un Étudiant s'inscrit à une ou plusieurs Formations.
- Une Formation est suivie par un ou plusieurs Étudiants.
- Une Inscription ne concerne qu'un seul Étudiant et une seule Formation.
- Un Paiement est lié à une seule Inscription.



1-2-Modèle Logique de Données (MLD)

Le MLD est la traduction du MCD en tables avec leurs attributs et les contraintes de clés primaires (PK) et étrangères (FK).



1-3-Schéma de la base de données

```
CREATE DATABASE IF NOT EXISTS inscriptionskfokam48;
```

```
-- On se positionne sur la base de données créée
```

```
USE inscriptionskfokam48;
```

```
-- TABLE: formations
```

```
CREATE TABLE IF NOT EXISTS formations (  
  id_formation INT PRIMARY KEY AUTO_INCREMENT,  
  nom_formation VARCHAR(160) NOT NULL,  
  description TEXT,
```

```
date_debut DATE NOT NULL,  
date_fin DATE NOT NULL,  
tarif DECIMAL(12,2) NOT NULL CHECK (tarif >= 0),  
nombre_places INTEGER NOT NULL CHECK (nombre_places >= 0)  
);
```

```
-- TABLE: etudiants
```

```
CREATE TABLE IF NOT EXISTS etudiants (  
id_etudiant INT PRIMARY KEY AUTO_INCREMENT,  
nom VARCHAR(100) NOT NULL,  
prenom VARCHAR(100) NOT NULL,  
date_naissance DATE NOT NULL,  
email VARCHAR(150) NOT NULL,  
telephone VARCHAR(30) NOT NULL,  
date_inscription DATE NOT NULL,  
CONSTRAINT uq_etudiants_email UNIQUE (email)  
);
```

```
-- TABLE: inscriptions
```

```
-- - FK explicites vers etudiants et formations  
-- - UNIQUE(id_etudiant, id_formation) pour éviter les doublons  
-- - statut_paiement contrôlé par ENUM  
--
```

```
CREATE TABLE IF NOT EXISTS inscriptions (  
id_inscription BIGINT PRIMARY KEY AUTO_INCREMENT,  
id_etudiant INT NOT NULL,  
id_formation INT NOT NULL,  
date_inscription DATE NOT NULL DEFAULT CURRENT_DATE,  
statut_paiement ENUM('En attente', 'Payé') NOT NULL DEFAULT 'En attente',  
CONSTRAINT uq_inscriptions_etud_form UNIQUE (id_etudiant, id_formation),  
CONSTRAINT fk_inscriptions_etudiant  
FOREIGN KEY (id_etudiant)  
REFERENCES etudiants(id_etudiant)  
ON UPDATE CASCADE ON DELETE RESTRICT,  
CONSTRAINT fk_inscriptions_formation  
FOREIGN KEY (id_formation)  
REFERENCES formations(id_formation)  
ON UPDATE CASCADE ON DELETE RESTRICT  
);
```

```
CREATE INDEX idx_inscriptions_formation ON inscriptions(id_formation);  
CREATE INDEX idx_inscriptions_etudiant ON inscriptions(id_etudiant);
```

```
-- TABLE: paiements
```

```
-- - FK explicite vers inscriptions
```

```
CREATE TABLE IF NOT EXISTS paiements (
id_paiement BIGINT PRIMARY KEY AUTO_INCREMENT,
id_inscription BIGINT NOT NULL,
montant DECIMAL(12,2) NOT NULL CHECK (montant > 0),
date_paiement TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
methode_paiement VARCHAR(40) NOT NULL,
CONSTRAINT fk_paiements_inscription
FOREIGN KEY (id_inscription)
REFERENCES inscriptions(id_inscription)
ON UPDATE CASCADE ON DELETE RESTRICT
);
-- TABLE: paiements_archive (historisation)
-- - Pas de FK pour ne pas bloquer l'archivage si l'origine est supprimée
```

```
CREATE TABLE IF NOT EXISTS paiements_archive (
id_archive BIGINT PRIMARY KEY AUTO_INCREMENT,
id_paiement BIGINT NOT NULL,
id_inscription BIGINT NOT NULL,
montant DECIMAL(12,2) NOT NULL,
date_paiement TIMESTAMP NOT NULL,
methode_paiement VARCHAR(40) NOT NULL,
archived_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

1-4-Documentation des tables

Table formations

Cette table stocke les informations sur les différentes formations proposées.

Champ	Type	Contraintes	Rôle
id_formation	INT	PRIMARY KEY, AUTO_INCREMENT	Identifiant unique de la formation.
nom_formation	VARCHAR(160)	NOT NULL	Le nom complet de la formation.
description	TEXT		Une description détaillée de la formation.
date_debut	DATE	NOT NULL	La date de début de la formation.
date_fin	DATE	NOT NULL	La date de fin de la formation.
tarif	DECIMAL(12,2)	NOT NULL, CHECK (tarif >= 0)	Le coût de la formation. La contrainte assure un tarif non négatif.
nombre_places	INT	NOT NULL, CHECK (nombre_places >=	La capacité d'accueil maximale de la formation. La contrainte

Champ	Type	Contraintes	Rôle
		0)	assure un nombre non négatif.

Table etudiants

Cette table contient les informations personnelles des étudiants.

Champ	Type	Contraintes	Rôle
id_etudiant	INT	PRIMARY KEY , AUTO_INCREMENT	Identifiant unique de l'étudiant.
nom	VARCHAR(100)	NOT NULL	Le nom de famille de l'étudiant.
prenom	VARCHAR(100)	NOT NULL	Le prénom de l'étudiant.
date_naissance	DATE	NOT NULL	La date de naissance de l'étudiant.
email	VARCHAR(150)	NOT NULL, UNIQUE	L'adresse email unique de l'étudiant.
telephone	VARCHAR(30)	NOT NULL	Le numéro de téléphone de l'étudiant.
date_inscription	DATE	NOT NULL	La date à laquelle l'étudiant a été enregistré dans le système.

Table inscriptions

Cette table gère la relation "plusieurs-à-plusieurs" entre les étudiants et les formations, enregistrant les inscriptions.

Champ	Type	Contraintes	Rôle
id_inscription	BIGINT	PRIMARY KEY , AUTO_INCREMENT	Identifiant unique de l'inscription.
id_etudiant	INT	NOT NULL, FOREIGN KEY vers etudiants(id_etudiant)	Référence à l'étudiant qui s'est inscrit.
id_formation	INT	NOT NULL, FOREIGN KEY vers formations(id_formation)	Référence à la formation à laquelle l'étudiant s'est inscrit.
date_inscription	DATE	NOT NULL, DEFAULT CURRENT_DATE()	La date de l'inscription.
statut_paiement	VARCHAR(20)	NOT NULL, DEFAULT 'En attente', CHECK IN ('En attente', 'Payé')	Le statut de paiement de l'inscription.
(Contrainte unique)		UNIQUE (id_etudiant, id_formation)	Empêche un étudiant de s'inscrire plus d'une fois à la même formation.

Table paiements

Cette table enregistre les transactions de paiement pour les inscriptions.

Champ	Type	Contraintes	Rôle
id_paiement	BIGINT	PRIMARY KEY , AUTO_INCREMENT	Identifiant unique du paiement.
id_inscription	BIGINT	NOT NULL, FOREIGN KEY vers inscriptions(id_inscription)	Référence à l'inscription à laquelle ce paiement est lié.
montant	DECIMAL(12,2)	NOT NULL, CHECK (montant > 0)	Le montant payé. La contrainte assure un montant strictement positif.
date_paiement	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP()	La date et l'heure du paiement.
methode_paiement	VARCHAR(40)	NOT NULL	La méthode de paiement utilisée (ex : carte bancaire, virement, etc.).

Table paiements_archive (Historisation)

Cette table est utilisée pour archiver les données des paiements, probablement pour des raisons de performance ou de conformité, sans interférer avec la table paiements active.

Champ	Type	Contraintes	Rôle
id_archive	BIGINT	PRIMARY KEY , AUTO_INCREMENT	Identifiant unique de l'enregistrement d'archive.
id_paiement	BIGINT	NOT NULL	Référence à l'identifiant du paiement archivé.
id_inscription	BIGINT	NOT NULL	Référence à l'identifiant de l'inscription liée au paiement.
montant	DECIMAL(12,2)	NOT NULL	Le montant du paiement archivé.
date_paiement	DATETIME	NOT NULL	La date et l'heure du paiement archivé.
methode_paiement	VARCHAR(40)	NOT NULL	La méthode de paiement archivée.
archived_at	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP()	La date et l'heure de l'archivage.

2. Scripts SQL

2-1-Scripts de création et d'insertion : confère
Projet2_Inscription/01_creation_et_insertion.sql

2-2-Scripts de requêtes : Projet2_Inscription/02_requetes.sql

2-3- Scripts de procédures et fonctions stockées :
Projet2_Inscription/03_fonction_et_procedure.sql

2-4-Scripts de vues et triggers : Projet2_Inscription/04_vues_et_triggers.sql

3-Rapport de Projet : Gestion des Inscriptions en Ligne pour la Formation KFOKAM48

3-1. Introduction

Ce rapport présente la conception et la mise en œuvre d'une base de données relationnelle nommée **inscriptionskfokam48**. L'objectif principal de ce projet était de créer un système robuste et structuré pour gérer les inscriptions à des formations, les informations sur les étudiants et les paiements associés. Le système est conçu pour assurer l'intégrité des données, prévenir les doublons et permettre une gestion efficace des processus d'inscription et de paiement.

3-2. Méthodologie

La méthodologie utilisée pour la conception de cette base de données a suivi plusieurs étapes clés, basées sur les principes de la modélisation relationnelle. .

- Analyse des Entités :

- **Formations** : Représentant les cours ou les programmes proposés.
- **Étudiants** : Contenant les informations personnelles des participants.
- **Inscriptions** : Liant les étudiants aux formations qu'ils suivent.
- **Paiements** : Enregistrant les transactions financières.
- **Paiements_archive** : Servant d'historique des paiements pour des raisons de conformité et de traçabilité.

- **Définition des Tables et des Attributs** : Pour chaque entité, on a créé une table avec des attributs pertinents. Les types de données ont été choisis pour optimiser le stockage et garantir la validité des informations (par exemple, DATE pour les dates, DECIMAL pour les montants financiers).

- **Gestion des Contraintes et des Clés** :

- **Clés primaires (PRIMARY KEY)** ont été définies pour identifier de manière unique chaque enregistrement (`id_formation`, `id_etudiant`, `id_inscription`, `id_paiement`).
- **Clés étrangères (FOREIGN KEY)** ont été établies pour lier les tables entre elles (`id_etudiant` et `id_formation` dans la table `inscriptions`, et `id_inscription` dans la table `paiements`). Ces relations sont cruciales pour maintenir la cohérence des données. La clause `ON UPDATE CASCADE ON DELETE RESTRICT` a été utilisée pour garantir que la suppression d'un enregistrement parent (comme une formation ou un étudiant) n'est pas possible s'il y a des inscriptions associées, évitant ainsi les liens brisés.
- **Contraintes d'unicité (UNIQUE)** ont été ajoutées pour empêcher les doublons, notamment l'unicité de l'adresse e-mail dans la table `etudiants` et l'unicité des paires (`id_etudiant`, `id_formation`) dans la table `inscriptions`.
- Des **contraintes de vérification (CHECK)** ont été mises en place pour assurer la validité des données numériques (`tarif` et `nombre_places` non négatifs, `montant` positif).
- Un **ENUM** a été utilisé pour le champ `statut_paiement` afin de limiter les valeurs possibles à 'En attente' et 'Payé', ce qui réduit les erreurs de saisie.

-**Optimisation des Performances** : Des **index** ont été créés (`idx_inscriptions_formation` et `idx_inscriptions_etudiant`) sur les clés étrangères les plus utilisées pour accélérer les recherches et les jointures de données.

- **Gestion de l'Histoire** : Une table `paiements_archive` a été conçue sans clés étrangères pour permettre l'archivage des enregistrements de paiements, même si les données source originales sont supprimées.

4. Résultats

a- Création de la base de données inscriptionsKFOKAM48 et des tables selon le modèle de données ci-dessus avec les contraintes de clé primaire et étrangère appropriées

Table	Action	Lignes	Type	Interclassement	Taille	Perte
☐ étudiants	★ Parcourir Structure Rechercher Insérer Vider Supprimer	0	InnoDB	utf8mb4_0900_ai_ci	32,0 kio	-
☐ formations	★ Parcourir Structure Rechercher Insérer Vider Supprimer	0	InnoDB	utf8mb4_0900_ai_ci	16,0 kio	-
☐ inscriptions	★ Parcourir Structure Rechercher Insérer Vider Supprimer	0	InnoDB	utf8mb4_0900_ai_ci	48,0 kio	-
☐ paiements	★ Parcourir Structure Rechercher Insérer Vider Supprimer	0	InnoDB	utf8mb4_0900_ai_ci	32,0 kio	-
☐ paiements_archive	★ Parcourir Structure Rechercher Insérer Vider Supprimer	0	InnoDB	utf8mb4_0900_ai_ci	16,0 kio	-
5 tables	Somme	0	InnoDB	utf8mb4_0900_ai_ci	144,0 kio	0 o

b-Insertion des données fictives dans la table Formations

	id_formation	nom_formation	description	date_debut	date_fin	tarif	nombre_places
☐ Éditer Copier Supprimer	1	DevOps Essentials	Introduction aux pipelines CI/CD et conteneurs	2025-10-01	2025-10-05	150000.00	30
☐ Éditer Copier Supprimer	2	Data Science 101	Fondamentaux de la data science	2025-10-10	2025-10-14	180000.00	25
☐ Éditer Copier Supprimer	3	Web Full-Stack	Front-end/Back-end modernes	2025-11-01	2025-11-08	200000.00	40

c-Requête pour lister tous les étudiants inscrits à une certaine formation

id_etudiant	nom	prenom	email	date_inscription	statut_paiement
1	Jordan	Nandjo	jordannandjo@yahoo.com	2025-09-09 00:00:00	En attente

d-Création une procédure stockée pour ajouter une nouvelle inscription

Nom	Type	Retourne
☐ prajouter_inscription	PROCEDURE	

e-Création d'un trigger pour mettre à jour le statut de paiement lors de l'ajout d'un paiement

```
UPDATE inscriptions SET statut_paiement = 'payé';
```

		id_inscription	id_etudiant	id_formation	date_inscription	statut_paiement
  Éditer	 Copier	 Supprimer	1	1	1 2025-09-09 00:00:00	Payé
 Éditer	 Copier	 Supprimer	2	2	3 2025-09-09 00:00:00	Payé

Conclusion

Le projet de base de données **inscriptionskfokam48** a abouti à un système bien structuré et fonctionnel qui répond aux besoins de gestion des inscriptions. La conception, basée sur une modélisation rigoureuse, garantit l'intégrité et la cohérence des données. Les contraintes et les index mis en place contribuent à la fiabilité et aux performances du système.

Une des principales difficultés rencontrées a été de s'assurer que les contraintes de clés étrangères et de vérification étaient correctement appliquées pour maintenir l'intégrité des données sans rendre le système trop rigide. Par exemple, le choix de **ON DELETE RESTRICT** sur la table **inscriptions** est un compromis qui empêche la suppression accidentelle d'informations critiques.